FinSight AI

Multi-Agent Stock Attention Engine

Architecture & Build Guide

Version 0.1 | September 2026

Prepared for internal reference and AI-assisted development continuity

1. What This System Is

This document describes the design and construction plan for the core reasoning engine of FinSight AI: a system that takes in a user's portfolio holdings, a temporal knowledge graph (TKG) of market and geopolitical events, live market data, and news/analyst sentiment, and produces a specific, evidence-backed answer to the question **"which stocks need my attention right now, and why?"**

This is explicitly a **triage and insight system, not a return-prediction (alpha) system**. It does not claim to predict which stocks will go up or down. It surfaces stocks that deserve a closer look — because they are underperforming, because a sector-level event is likely to affect them, or because specialist signals about them disagree — and explains the reasoning with evidence. This framing is intentional and matches FinSight's regulatory positioning as an insight/education layer rather than personalized investment advice.

2. Guiding Design Principles

- **Code over LLM wherever possible.** Pure computation — portfolio weights, returns, volatility, drawdown — is done in Python (pandas/numpy), never asked of an LLM. LLM calls are reserved for reasoning over qualitative, messy, or causal information that code cannot do.
- **Scope every agent tightly.** No agent ever reasons over the full market. The TKG agent only traverses the sectors the user actually holds; the sentiment agent only aggregates news for held tickers/sectors. This is both a cost control and a quality control.
- **Two-stage funnel.** A cheap, broad screen (Stage 1) runs over the entire portfolio. An expensive, detailed deep-dive (Stage 2) runs only on the resulting shortlist.
- **Variable-size shortlist.** The number of stocks flagged is threshold-based, not a fixed count. Any stock that crosses a risk or opportunity threshold on any combination of signals is included — there is no fixed "top 10."
- **Disagreement is itself a signal.** When the fundamental, valuation, and sentiment reads on a stock conflict, that conflict is surfaced and explained rather than averaged away.
- **Escalate only when needed.** The expensive conflict-resolution LLM step runs only for stocks where structured signals disagree — not by default for every stock.
- **Compliance is mandatory, not optional.** Every output passes through a framing/compliance pass before reaching the user, given FinSight's SEBI RIA positioning.
- **Nothing is trusted until backtested.** The architecture being logically sound does not mean its predictions are accurate. Historical backtesting against real portfolios and market outcomes is a required step before this system is relied upon, and comes after the full pipeline is constructed.

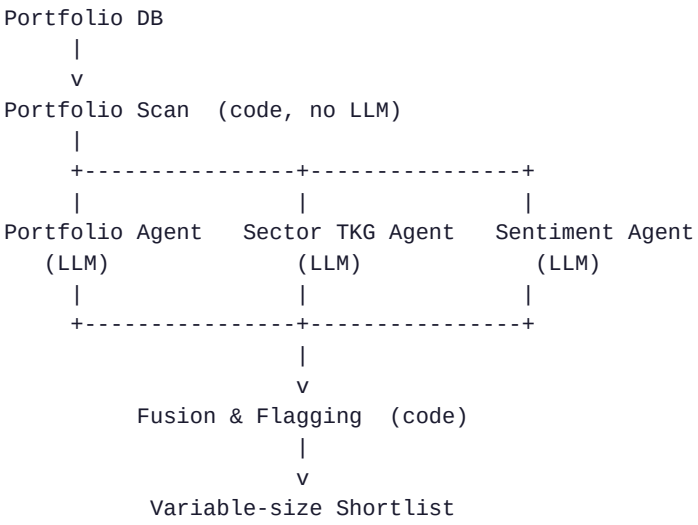
3. Architecture Overview

3.1 Stage 1 — Screening (runs over the full portfolio)

A cheap, code-only **Portfolio Scan** queries the portfolio database and computes holdings, sector exposure, and structural performance metrics — no LLM call, no token cost. This output scopes three agents that then run in parallel:

- **Portfolio Agent** (LLM) — narrates the structural weak/strong read from the scan.
- **Sector TKG Agent** (LLM + graph query) — queries the event store for the user's held sectors only, and reasons over event-to-impact chains (e.g. an export ban affecting a raw material affecting a held company).
- **Sentiment Agent** (LLM + FinBERT) — aggregates analyst ratings and FinBERT-scored news sentiment for held tickers/sectors, and narrates direction and magnitude.

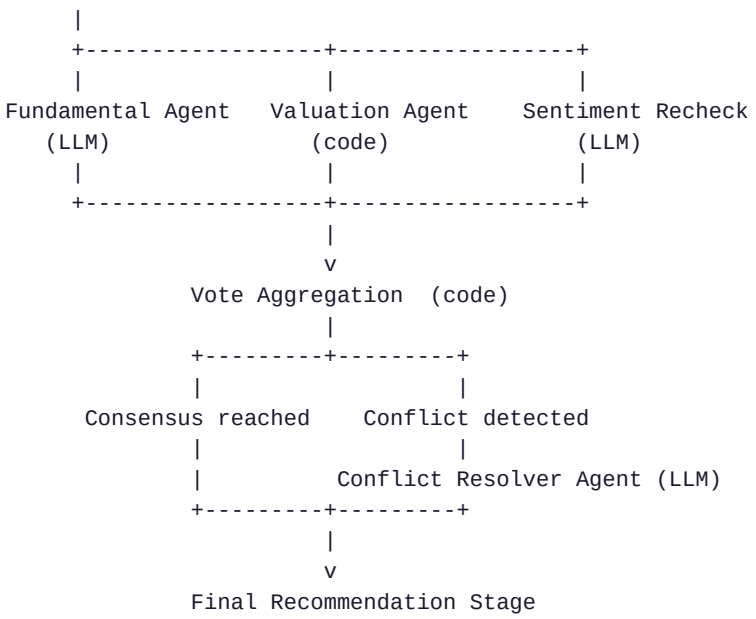
Their outputs converge into a deterministic **Fusion & Flagging** step (code, not LLM) that cross-references all three signals per stock and produces the variable-size shortlist: underperformers, at-risk holdings, and opportunities. This is the step that catches the case a single agent would miss — a stock that looks structurally fine but carries negative sector-event exposure.



3.2 Stage 2 — Deep Dive (runs only on the shortlist)

- **Fundamental Agent** (LLM) — earnings, growth, and balance sheet health, per shortlisted stock.
- **Valuation Agent** (code) — historical price/volume → returns, volatility, drawdown, technical signals, computed in pandas/numpy. The LLM only narrates these numbers into evidence; it never computes them.
- **Sentiment Recheck** (LLM + FinBERT) — sentiment re-scoped tightly to news about this individual stock, not the whole sector.

These three emit a short structured verdict each: `{stock, action, confidence, rationale}`. A deterministic **Vote Aggregation** step (code) computes a weighted consensus score across the three and detects conflict. Only when the votes genuinely disagree does a **Conflict Resolver Agent** (LLM) get invoked, specifically to explain the disagreement for that stock — this conditional escalation is what keeps the cost of this stage bounded.



3.3 Final Recommendation Stage

- **Final Synthesis Agent** (LLM) — compiles all collected evidence (Stage 1 + Stage 2 + any conflict resolution) into the user-facing recommendation text.
- **Compliance / Framing Agent** (LLM + rules) — reframes directive language into insight/education framing consistent with SEBI RIA positioning, and attaches the standard disclaimer.
- The compiled graph is exposed via a FastAPI endpoint (e.g. `POST /analyze/{user_id}`) that triggers one full run and returns the structured, evidenced output to the frontend.

4. Technology Stack & Rationale

Component	Technology	Why
Orchestration	LangGraph	Explicit shared state + conditional edges; avoids unbounded agent-to-agent chatter and keeps token cost predictable.
LLM core	Claude API	Reasoning calls only, where qualitative/causal judgment is genuinely required.
Backend	FastAPI	Matches existing FinSight stack; exposes the compiled graph as an endpoint.
Structured DB	PostgreSQL	Holdings, transactions, goals — ACID-compliant financial data.
Event / TKG store	Postgres tables first, Neo4j later	Start simple; graduate to a graph database once genuine multi-hop reasoning is needed.
Sentiment scoring	FinBERT (HuggingFace)	Free, pretrained, reliable numeric sentiment scoring — no fine-tuning required.
News source	NewsAPI / RSS feeds	Broad financial and geopolitical news coverage.
Market data	Polygon.io / NSE API	Price and volume history feeding the Valuation agent.
Observability	LangSmith	Full node-level tracing so the pipeline stays inspectable rather than an actual blackbox.

5. Build Blueprint — Eight Phases

Built bottom-up: deterministic, LLM-free code first, so the data plumbing is proven correct before any tokens are spent. Each phase should be tested in isolation before being wired into the next.

1. Define the state schema

Before writing a single agent, define one Pydantic model representing everything that will ever flow through one analysis run — holdings, sector exposure, each agent's output, the shortlist, votes, final text. Every node reads from and writes to this object. Spend real time here: reworking a schema field later is cheap, reworking the whole data flow isn't.

2. Build the deterministic core first, zero LLM calls

Write Portfolio Scan (queries Postgres, computes weights, sector exposure, structural metrics), Valuation (given price/volume history, computes returns, volatility, drawdown in pandas/numpy), and Fusion (combines signal scores into flags) as plain Python functions. Test each against known inputs. This produces a free, working skeleton before a single token is spent, and proves the plumbing is correct.

3. Stand up minimal versions of the two external data sources

The TKG agent needs an event store to query; the sentiment agent needs scored news. Don't build a graph database yet — start with a couple of Postgres tables (events, sector_impact) and a simple NewsAPI-to-FinBERT pipeline scoring each article. Graduate the event store to Neo4j later, once genuine multi-hop reasoning (event to commodity to company) is actually needed.

4. Build and test each Stage 1 agent in isolation

Write the three LLM-backed nodes: Portfolio Agent (narrates the scan's structural output), Sector TKG Agent (queries the event store scoped to held sectors and reasons over what it finds), and Sentiment Agent (narrates the FinBERT aggregate). Call each directly with realistic test input and read the output before it becomes part of a bigger pipeline — this is the cheapest point to catch a bad prompt.

5. Wire Stage 1 into LangGraph and test the fan-out/fan-in

This is the first real LangGraph code. Define each function from phases 2 and 4 as a node, and connect them with edges: Portfolio Scan feeds all three agents in parallel, and their outputs converge into Fusion. Run this subgraph end-to-end on one real test portfolio and sanity-check the shortlist before adding anything else.

6. Build Stage 2: deep dive, voting, and conditional escalation

Add Fundamental Agent (given financial statement data from a screener API), reuse the Valuation function per shortlisted stock, and reuse the sentiment pipeline scoped to one ticker. Add a Vote Aggregation function (code) that scores agreement or conflict across the three. Then write the first conditional edge — it inspects the vote result and routes only conflicting stocks to a Conflict Resolver agent, skipping it entirely when the signals already agree.

7. Final synthesis, compliance pass, and the FastAPI wrapper

Add a Final Synthesis agent that turns all collected evidence into the user-facing recommendation, and a Compliance agent that reframes it as insight/education and attaches the disclaimer. Wrap the compiled graph in a FastAPI endpoint, e.g. POST /analyze/{user_id}, that triggers one run and returns the structured result to the frontend.

8. Add tracing, then move to backtesting and cost tuning

Before trusting any output, wire in LangSmith so every node's input and output is inspectable — this is what keeps a multi-agent system from actually being a blackbox. Only once the full pipeline runs reliably end-to-end does backtesting against historical portfolios begin, and only after that comes cost optimization: Haiku for Stage 1 vs. Sonnet for Stage 2, caching sector-level TKG results, batching calls.

6. Design Clarifications Log

Recorded answers to questions that came up while locking this design, kept here so the reasoning behind each decision doesn't have to be re-derived later.

Why call this "multi-agent" if much of it is plain code?

Because the parts that matter for the system's judgment — TKG causal reasoning, sentiment interpretation, fundamental narration, conflict resolution, and final synthesis — are genuinely agentic in the sense of being specialized, context-heavy LLM reasoning components, not a generic chatbot prompted once with everything attached. The label doesn't require every node to call external tools autonomously; it's fine to call this a multi-agent architecture as long as it's clearly understood which nodes are LLM reasoning and which are deterministic code.

What does the vote aggregation stage actually do, and is it required?

Only the three Stage 2 deep-dive outputs vote (fundamental, valuation, sentiment recheck) — Stage 1 agents don't get a second vote. Mechanically it is a deterministic weighted scoring function, not a separate deliberating agent. Its value is as a disagreement detector: when the three signals agree, that's a clean high-confidence read; when they conflict, the conflict itself is the interesting finding (e.g. a "value trap" or "priced-for-perfection" case) and is surfaced explicitly rather than averaged away. This was judged worth keeping for the added sophistication it gives the platform's output.

Is the multi-agent approach correct, and is this specific structure correct?

Decomposing the reasoning by specialty (structured portfolio data, graph/event data, unstructured news, price time series) is a sound and standard engineering pattern for this kind of heterogeneous problem. Whether this specific structure reliably flags the right stocks is a separate, empirical question that no architecture diagram can answer on its own — that is what the backtesting phase (Phase 8) exists to establish. The system should be treated as unvalidated until that backtest has run.

What is this system actually predicting?

Attention and risk triage — which stocks deserve a closer look and why, grounded in real signals, plus deeper insight into how global or sector-level events might directly or indirectly affect the user's holdings. It does not predict future returns or generate alpha, and it is not framed as personalized investment advice, consistent with FinSight's SEBI RIA positioning.

7. Data Flow Summary — What Connects to What

- PostgreSQL (portfolio, goals) → Portfolio Scan node
- Event/TKG store (Postgres tables, later Neo4j) → Sector TKG Agent
- NewsAPI + FinBERT scores → Sentiment Agent and Sentiment Recheck
- Market data API (Polygon.io / NSE) → Valuation Agent
- All Stage 1 agent outputs → Fusion & Flagging → Variable-size Shortlist
- Shortlist → Stage 2 agents (Fundamental, Valuation, Sentiment Recheck)
- Stage 2 outputs → Vote Aggregation → conditional edge → optional Conflict Resolver Agent
- All collected evidence → Final Synthesis Agent → Compliance Agent → FastAPI response → frontend

8. Next Step

Begin construction at Phase 1: define the shared state schema. Everything else in this guide builds on top of that contract.